

Fairness-centric Global Placement in VLSI Physical Design

Wai-Shing Luk

Abstract—Routing congestion has become the dominant concern in modern VLSI physical design, yet traditional global placement objectives such as total wirelength or net-cut minimization do not guarantee a uniform distribution of routing demand and can leave locally congested regions. This paper argues for **fairness-centric global placement**: rather than minimizing an aggregate cost, the placer should minimize the worst-case congestion and equalize routing demand across the chip. We review the limitations of wirelength-driven placement, define congestion and its overflow metric, introduce the max-min fairness principle from network resource allocation, and outline objective functions and algorithmic techniques that realize it. We then discuss the interplay with wirelength, timing, and power objectives and identify open challenges in complexity, congestion estimation, mixed-size placement, multi-layer routing, and benchmarking. We then describe an open-source implementation that realizes the objective with Howard’s parametric minimum-cost-flow algorithm and bipartite-matching legalization, and we report placement and congestion-map results on a standard benchmark.

Index Terms—VLSI physical design, global placement, routing congestion, max-min fairness.

I. INTRODUCTION

The automated placement of circuit components, or cells, onto a silicon die is a critical step in the design of Very Large Scale Integrated (VLSI) circuits. The quality of the placement significantly impacts various aspects of the final integrated circuit, including its area utilization, power consumption, timing performance, and, crucially, its routability. Traditional objectives in global placement have often centered around minimizing total wirelength or net-cut cost, aiming to reduce interconnect delays and area. However, with the increasing complexity and density of modern VLSI designs, the problem of routing congestion has emerged as a paramount concern.

Routing congestion arises when the demand for routing resources in certain regions of the chip exceeds the available supply, leading to routing detours, increased wirelength, potential timing violations, and even unroutable designs. While minimizing total wirelength aims to reduce the overall wiring demand, it does not guarantee a uniform distribution of this demand across the chip. In fact, a placement with globally minimized wirelength can still exhibit highly congested local regions, as the algorithm may prioritize shortening some nets at the expense of others, leading to an uneven distribution of routing resources.

This inherent limitation of traditional placement objectives has motivated the exploration of congestion minimization as a primary goal during placement. Among various approaches to tackle congestion, the concept of **fairness** in the allocation of routing resources has gained increasing attention. This paper argues for a shift towards **fairness-centric global placement** in VLSI physical design. We will explore the limitations of solely focusing on global minimization objectives like wirelength, the motivation behind adopting a fairness perspective, the principles of fair resource allocation, potential approaches to implement fairness-centric placement, and the challenges and future directions in this domain. By drawing parallels with resource management in other fields and analyzing existing techniques, this paper aims to provide a comprehensive understanding of the importance and potential of fairness-centric methodologies in achieving routable and high-quality VLSI designs.

Beyond surveying the motivation and principles, this paper grounds the discussion in a working system. Following the survey, we present a concrete fairness-centric placer, the “No-Nonsense” (NNS) algorithm; show how its min-max objective reduces to a parametric minimum-cost-flow problem solved by Howard’s algorithm; examine the congestion maps it induces; evaluate it on a standard benchmark; and sketch how the resulting placement flows into global routing.

II. BACKGROUND

A. VLSI Physical Design Flow and Placement

The physical design of a VLSI circuit is a complex process that transforms a circuit netlist into a geometric layout ready for fabrication [1], [2]. This process is typically broken down into several stages, including floorplanning, placement, routing, and physical verification. **Placement** is the stage where physical locations are determined for all the components (standard cells, macros, IP blocks) of the circuit within the defined core area of the chip. The quality of the placement lays the foundation for the subsequent routing stage and significantly influences the overall performance and manufacturability of the final chip.

Placement is often performed in two phases: **global placement** and **detailed placement**. Global placement aims to find approximate locations for all cells, optimizing a given objective function (e.g., wirelength, congestion) while considering overall chip dimensions and potential placement blockages. During this phase, cells may overlap.

Detailed placement then refines the global placement by removing cell overlaps and assigning cells to legal locations on the placement grid, often involving further optimization steps like cell swapping or shifting.

B. Traditional Global Placement Objectives and Limitations

Historically, global placement has primarily focused on minimizing **total wirelength**, which is often estimated using metrics like half-perimeter wirelength (HPWL) [1], [2]. The rationale behind this objective is that shorter wires generally lead to lower interconnect delay, reduced power consumption, and potentially smaller chip area. Various analytical and combinatorial techniques have been developed for wirelength-driven global placement, including quadratic programming, simulated annealing, and partitioning-based methods.

Another traditional objective is minimizing **net-cut**, particularly in partitioning-based placement algorithms. Net-cut refers to the number of signal nets that cross the boundaries of partitions during the placement process. Minimizing net-cut aims to reduce the number of long interconnections and improve locality [3].

However, as VLSI design complexity has escalated, the limitations of solely relying on wirelength or net-cut minimization have become increasingly apparent, especially concerning **routing congestion**. While reducing global wirelength can help minimize the total wiring demand on the chip, it does not guarantee a uniform distribution of this demand. An algorithm optimizing for total wirelength might cluster highly interconnected cells together in a small region, leading to a high density of routing demand in that area, even if the overall wirelength is minimized. Conversely, other regions might have abundant routing resources that remain underutilized.

As highlighted in the sources, “**minimizing wirelength may (and in general, will) create locally congested regions**”. It is entirely possible for a minimum wirelength solution to require more routing resources through a particular region than are physically available. This excessive congestion can severely hinder the subsequent routing stage, potentially leading to a larger final routed wirelength due to detours around congested areas, increased routing complexity, longer design cycle times due to iterations between placement and routing, and even unroutable designs in fixed-die regimes.

C. Wirelength Models: Quadratic Placement and HPWL

The cost model, not the optimizer, is where much of the accidental complexity of placement lives. The **quadratic model** expresses a net’s cost as a sum of squared pin-to-pin distances and turns placement into a sparse linear system [4]; it is the engine inside GORDIAN and its analytic descendants [5], [6]. Squared Euclidean distance, however, matches neither the Manhattan geometry in which wires are actually routed nor the concave wire costs of a fixed FPGA

fabric [7], [8]. Quadratic placers therefore add pseudo-nets and pseudo-IO points to correct the model’s artifacts, exchanging mathematical simplicity for implementation complexity [9]. Separating this *accidental* complexity from the *essential* complexity of placement – the non-convex non-overlap constraints and the wirelength-versus-congestion trade-off – is the central design skill [10].

The standard physical metric is the **half-perimeter wirelength (HPWL)**. For a net e , it is half the perimeter of the smallest axis-aligned bounding box that encloses the net’s pins [1], [2]:

$$\text{HPWL}(e) = (\max_{v \in e} x_v - \min_{v \in e} x_v) + (\max_{v \in e} y_v - \min_{v \in e} y_v).$$

In Manhattan routing this is a cheap and tight lower bound on the routed wire length, and unlike squared distance it is linear in the coordinates. It is, however, piecewise linear and therefore **non-differentiable** wherever two pins tie for a bounding-box edge – exactly the regime in which gradient-based analytic placers operate [11].

To restore differentiability, modern placers replace the max and min in HPWL with smooth surrogates: the **log-sum-exp (LSE)** function introduced for placement by Naylor et al. [12], and the **weighted-average (WA)** form used by APlace and ePlace [13], [14]. Both approach HPWL as their smoothing parameter is tightened, and both appear in the leading analytical placers – FastPlace, SimPL, NTUPlace3, and RePLace [6], [9], [11], [15]; multilevel mixed-size placers such as mPL6 use the same smoothed objective together with explicit congestion control [16].

HPWL and its smooth approximations remain **sums over nets**. They estimate how much wire exists, but not where it is spent: a placement with a small total HPWL can still route most of that demand through a single crowded region. The gap between a good aggregate and a good distribution is precisely what the fairness objective of this paper is designed to close.

D. A Four-Decade History of Global Placement

Global placement has been studied for more than forty years, and its objectives and algorithms have evolved in several distinct waves. The first wave was **partitioning- and min-cut-based placement**: Breuer’s min-cut placer [17] and the graph-partitioning heuristics of Kernighan and Lin [18] and Fiduccia and Mattheyses [3] cut a circuit into halves that were assigned to opposite sides of the die, an idea later extended to full standard-cell placement by Dunlop and Kernighan [19]; the survey of Alpert and Kahng [20] collects this line of work. A second wave applied **simulated annealing** to placement, trading runtime for solution quality: the canonical annealing formulation [21] and its influential circuit implementation, TimberWolf [22], made annealing the reference placer of the late 1980s.

A third, ultimately dominant, wave is **analytical placement**, which models placement as continuous optimization and rounds the continuous result. Hall’s quadratic placement [4] minimized squared wirelength; Goto’s method [23] and GORDIAN [5] combined quadratic programming with partitioning or slicing; the force-directed formulation of Quinn and Breuer [24] belongs to the same family; and Eisenmann and Johannes [25] showed that a generic analytic engine can handle placement and floorplanning together. Early timing-driven methods tuned these engines toward critical paths [26], and recursive bisection was shown to be competitive for routability as well [27]. Today’s analytical placers continue this lineage: FastPlace [9], APlace [13], NTUPlace3 [6], SimPL [11], the electrostatics-based ePlace [14], and RePLace [15], while a parallel effort replaced the exponential-time annealing engine with efficient linear-wirelength solvers [28].

The most recent wave is **GPU acceleration and machine learning**. DREAMPlace [29] recasts analytic placement as a deep-learning training problem in order to run on GPUs, and ABCDPlace [30] accelerates the detailed-placement stage in the same way. Reinforcement learning has been applied to chip and macro placement [31], and hierarchical macro placers now handle large IP blocks [32]; recent surveys map the broader machine-learning-for-EDA landscape [33], [34]. **Field-programmable gate arrays**, with their fixed routing fabric and irregular columns, have driven a parallel line of placement research: from the annealing-based VPR [7] and its timing-driven extension [35], through analytic placers for heterogeneous FPGAs [8], [36] and routability-driven FPGA placement [37] within the open VTR flow [38], to reinforcement-learning placers [39].

Across these four decades the objective has almost always been an aggregate – total wirelength, total cut, or total overflow. That choice is what leaves room for a fairness-centric formulation.

E. Defining and Measuring Congestion

Congestion in VLSI layout intuitively refers to a situation where too many nets need to be routed in localized areas, leading to a shortage of routing resources. To quantify and manage congestion during placement, the concept of a **global bin grid** is commonly employed. The chip area is divided into a grid of rectangular regions called global bins, and the boundaries between these bins are referred to as global bin edges.

The **routing demand** (d_e) on a global edge (e) is defined as the number of routed nets that cross that edge. This demand is estimated based on the cell placement using a global router or even a simpler model like a bounding box router. The **routing supply** (s_e) of a global edge is the physical routing capacity available across that edge, which is determined by the length of the edge and the technology parameters (e.g., number of routing layers, wire pitch).

A global edge is considered **congested** if the routing demand exceeds the routing supply ($d_e > s_e$). The **overflow** ($overflow_e$) of a congested edge is the amount by which the demand exceeds the supply: $overflow_e = d_e - s_e$ if $d_e > s_e$, and 0 otherwise. The **total overflow** of a placement is the summation of the overflows across all global edges and serves as a global measure of congestion. A placement with a lower total overflow is generally considered less congested. Industry experience suggests that total overflow is a good indicator of overall routability. Congestion maps generated by CAD vendors often visualize this overflow information, highlighting congested regions.

Congestion has long been attacked at placement time. Classical frameworks penalize estimated overflow during analytical placement [40], reserve or redistribute white space to relieve dense regions [41], or move cells to relieve hotspots after an initial placement [42]; routability-driven contests subsequently made overflow a standard, comparable benchmark metric [43].

F. Relationship Between Wirelength and Congestion

Intuitively, a placement with optimized wirelength is expected to have fewer nets traversing the same region, thus potentially leading to lower congestion. As stated in one source, “**a layout with optimized wirelength will have less nets going through the same region, thus the congestion cost of the layout is also expected to be minimized**”.

Furthermore, there is a fundamental relationship between the total wirelength and the total routing demand in a global placement context. If cells are assumed to be placed at the centers of global bins and the wirelength is measured in units of global bin dimensions, then “**the total wirelength of a global placement is equal to the total routing demand on all global edges**”. This is because each unit of wirelength in this discretized space corresponds to a crossing of a global bin edge, contributing one unit to the routing demand.

This observation highlights that minimizing wirelength does indeed minimize the total amount of routing demand on the chip, and thus the average routing demand on each global edge. Given a fixed routing supply, reducing the average demand increases the likelihood of achieving a low-congestion layout globally. Based on this, it can be concluded that “**minimizing congestion is globally consistent with minimizing wirelength**”.

However, this global consistency does not necessarily translate to local consistency. As demonstrated by an example in one source, a placement optimized for congestion (even distribution of nets) can have a higher total wirelength than a placement optimized for wirelength (clustering connected cells), which in turn exhibits local congestion. This illustrates that while wirelength minimization helps reduce the overall routing pressure, it does not prevent the formation of local hotspots where the routing demand

exceeds the available supply. Therefore, traditional placement schemes based primarily on wirelength minimization “cannot adequately account for congestion”.

III. THE PROBLEM OF CONGESTION IN VLSI PLACEMENT

A. Local Congestion Despite Global Wirelength Minimization

As previously discussed, the primary limitation of wirelength-driven placement is its inability to directly address the uniform distribution of routing resources. Algorithms focused on minimizing the sum of wirelengths across all nets may inadvertently create areas with a high density of interconnected cells and their corresponding routing demands. This clustering effect, while beneficial for reducing the overall wirelength, can lead to local congestion where the number of nets attempting to pass through a particular region exceeds the routing capacity of that region.

One source provides a compelling example of this dichotomy. A circuit with eight cells and four circularly connected nets is considered. A congestion-optimal placement evenly distributes the nets across the chip, resulting in zero overflow (a routable solution), but potentially a longer total wirelength. Conversely, a wirelength-optimized placement groups the interconnected cells closely together, leading to a shorter total wirelength but causing an overflow on a global edge due to the concentrated routing demand. This simple example effectively illustrates that “**minimizing congestion is not equivalent to minimizing wirelength**”.

This trend is not limited to small examples but has also been observed in the placement of large industrial circuits. Congestion maps of wirelength-optimal placements often reveal significant localized congestion. This uneven distribution of routing demand can create bottlenecks for the subsequent routing algorithms.

B. Detrimental Effects of Congestion

Excessive routing congestion has several detrimental effects on the VLSI design process and the quality of the final product:

- **Routability Issues:** The most direct consequence of congestion is the difficulty in completing the routing of all the nets. When the routing demand exceeds the supply in certain areas, the router may be forced to take long detours around these congested regions to find available routing tracks. In severe cases, congestion can lead to an **unroutable placement** in fixed-die scenarios where the chip area is fixed.
- **Increased Wirelength:** Routing detours caused by congestion inevitably lead to an increase in the final routed wirelength, even if the initial placement had a relatively short estimated wirelength. This increased wirelength can negatively impact circuit

performance due to increased interconnect delay and power consumption.

- **Performance Degradation:** Longer interconnects resulting from routing detours contribute to increased propagation delays, potentially leading to timing violations and a reduction in the overall performance of the circuit.
- **Increased Chip Area:** In cases where a fixed-die size is not a strict constraint, severe congestion might necessitate an increase in the chip area to provide more routing resources, leading to higher manufacturing costs.
- **Manufacturing Yield Issues:** Highly congested designs can also be more susceptible to manufacturing defects due to the increased density of wires and vias.
- **Longer Design Cycle Times:** Iterations between the placement and routing stages become more frequent and time-consuming when significant congestion is present. The router’s inability to complete the routing might necessitate revisiting the placement and performing adjustments, prolonging the overall design cycle.
- **Global Router Performance Degradation:** Congested areas can also negatively impact the performance and efficiency of global routers, making it harder for them to find optimal routing paths.

C. Increasing Importance in Advanced Technology Nodes

The problem of routing congestion has become increasingly critical with the advancement of VLSI technology and the shrinking feature sizes. As technology nodes decrease, the number of transistors that can be integrated onto a single chip increases dramatically, leading to more complex designs with a higher density of cells and interconnections. Simultaneously, the routing resources (number of routing layers, wire pitch) may not scale at the same rate, leading to a tighter constraint on the available routing capacity.

Furthermore, in modern designs, especially Field-Programmable Gate Arrays (FPGAs) with fixed routing resources, timing and congestion concerns often outweigh total wirelength. The fixed routing architecture of FPGAs makes them particularly sensitive to congestion, as routing detours can severely impact performance and may lead to designs that cannot be implemented within the available resources.

Therefore, addressing and mitigating routing congestion during the placement phase has become an indispensable aspect of modern VLSI physical design to ensure successful routing, achieve desired performance targets, and optimize overall design quality.

IV. FAIRNESS-CENTRIC GLOBAL PLACEMENT: MOTIVATION AND PRINCIPLES

A. The Need for Fair Resource Allocation

The limitations of traditional global placement objectives in adequately addressing routing congestion stem from

their focus on global minimization (e.g., total wirelength) without explicitly considering the **fairness** in the distribution of the resulting routing demand across the available resources. Just as urban planners need to manage city development by not only minimizing the total distance between amenities but also ensuring that resources are reasonably accessible to all residents, placement engineers need to organize components in a way that avoids creating localized resource scarcity (congestion) even if the overall resource consumption (total wirelength) is low.

One source aptly captures this challenge by citing a Chinese proverb: “**we do not worry about scarcity, but about unfairness**”. In the context of VLSI routing, this implies that the primary issue is not necessarily an overall lack of routing resources, but rather their unequal distribution, leading to congestion in some areas while others remain underutilized. This “unfair” allocation of routing demand can lead to the problems discussed in the previous section.

B. Max-Min Fairness Principle

To address this issue of unequal resource distribution, the principle of **max-min fairness** offers a promising paradigm for global placement. This principle, originating from the field of communication networks and network traffic management [44], [45], aims to **maximize the minimum resources allocated to each agent**, ensuring a baseline level of service while allowing for flexible allocation beyond that minimum. In the context of bandwidth allocation in networks, max-min fairness ensures that no flow can have its rate increased without decreasing the rate of another flow that has an equal or smaller rate. This concept provides a perfect parallel to the problem of routing resource allocation in chip design.

C. Application to Placement

In the context of global placement, applying the max-min fairness principle translates to minimizing the worst-case routing congestion (or overflow) across all regions of the chip. Instead of solely focusing on minimizing the total overflow, a fairness-centric approach prioritizes ensuring that no single region experiences excessively high congestion, even if this might lead to a slightly higher total wirelength compared to a purely wirelength-optimized placement. The goal is to achieve a more balanced distribution of the routing demand, making it easier for the subsequent routing algorithms to find feasible paths for all nets without resorting to significant detours or encountering unroutable situations.

This shift in optimization objective, from global minimization to prioritizing fairness by minimizing the maximum congestion, fundamentally changes the approach to global placement. It acknowledges that in modern VLSI designs with tight routing resource constraints, especially in FPGAs, achieving a certain level of local routability across the entire chip is often more critical than achieving the absolute minimum total wirelength.

D. Contrasting with Traditional Objectives

Traditional global placement objectives, like minimizing total wirelength, are essentially global minimization problems. While they aim for an optimal solution across the entire chip, they do not inherently prevent localized problems like high congestion. An algorithm aggressively minimizing total wirelength might make placement decisions that significantly increase the routing demand in certain already crowded areas, sacrificing the “fairness” of resource distribution for a better global cost.

In contrast, a fairness-centric approach directly addresses the uneven distribution of routing demand by focusing on the regions with the highest congestion. By attempting to reduce the peak congestion levels, these methods aim to improve the overall routability and quality of the final layout, potentially leading to better performance and manufacturability even if the total wirelength is slightly higher than what a purely wirelength-driven placer might achieve. The focus shifts from achieving the absolute best global metric to ensuring a more balanced and manageable routing landscape across the chip.

V. APPROACHES TO FAIRNESS-CENTRIC GLOBAL PLACEMENT

Implementing a fairness-centric approach to global placement requires modifying the objective function and/or the algorithmic techniques used during the placement optimization process. Several potential strategies can be explored:

A. Modifying the Objective Function

The objective function in global placement guides the optimization process. To incorporate fairness, the objective can be modified to:

- **Minimize the Maximum Congestion:** Instead of minimizing the total overflow, the objective can be to minimize the maximum overflow observed on any global bin edge. This directly targets the “worst-case” congestion scenario and encourages a more even distribution of routing demand.
- **Weighted Congestion Minimization:** The congestion cost for each global bin edge can be weighted based on its current congestion level. Edges with higher congestion would have a larger weight, making the optimization algorithm more sensitive to reducing congestion in these critical areas.
- **Hybrid Objectives with Fairness Terms:** The traditional wirelength objective can be combined with a term that quantifies the “unfairness” in congestion distribution. This term could, for example, be the variance of the congestion levels across all global bin edges. The optimization would then aim to minimize a weighted sum of wirelength and this unfairness metric, allowing for a trade-off between global wirelength and local congestion balance. However, as noted earlier, simple hybrid length plus congestion objectives have

not always proven very effective, suggesting that the formulation of such hybrid objectives for fairness might require careful consideration.

B. Algorithmic Techniques for Achieving Fairness

Various algorithmic techniques can be adapted or developed to achieve fairness in global placement:

- **Iterative Refinement with Congestion Feedback:** Global placement can be performed in iterations, with congestion analysis performed after each iteration. Highly congested regions can be identified, and subsequent iterations can focus on moving cells out of these areas and into less congested regions. This requires effective mechanisms for identifying and alleviating congestion hotspots. **Post-processing techniques** specifically designed for congestion minimization after an initial placement, such as **net-centric approaches** that try to reroute congested nets by moving connected cells, have also shown promise.
- **Network Flow Based Methods:** Network flow formulations can model the routing demand and capacity constraints in the global bin grid. By incorporating cost functions that penalize exceeding the capacity of edges (congestion), these methods can implicitly encourage a fairer distribution of flow (routing demand). Furthermore, max-flow min-cut theorem and related concepts might offer insights into identifying and resolving congestion bottlenecks.
- **Mathematical Programming with Fairness Constraints:** Linear programming (LP) or integer programming (IP) formulations of the global placement problem can include constraints that directly limit the congestion on each global bin edge or minimize the maximum congestion. While these methods can be computationally expensive for very large designs, they offer the potential for directly optimizing fairness metrics.
- **Legalization-Assisted Placement:** Integrating legalization constraints earlier in the placement process can help prevent severe overlaps and potentially reduce the likelihood of high congestion in later stages. By maintaining a more spread-out placement throughout the optimization, this approach can indirectly contribute to a fairer distribution of routing demand.
- **Cell Shifting and Spreading Techniques:** Techniques like cell shifting, where cells are moved based on bin utilization to reduce overlap, and spreading forces, which prevent cells from collapsing back into dense regions, can contribute to a more even cell distribution, which in turn can lead to a fairer distribution of routing demand.

C. Empirical Evidence and Existing Approaches

While the explicit term “fairness-centric” might not be universally used, several existing congestion-driven placement techniques implicitly aim for a fairer distribution of routing resources:

- **Congestion-Driven Placement Based on Multi-Partitioning:** These methods use actual congestion cost calculated from pre-computed Steiner trees to minimize congestion, aiming to distribute nets more evenly across partitions.
- **Congestion-Driven Quadratic Placement:** These approaches incorporate congestion metrics into the quadratic objective function, penalizing placements that lead to high congestion.
- **Algorithms Using Congestion Maps to Guide Optimization:** Many modern placers use congestion maps, generated through static, probabilistic, or constructive route estimation, to identify congested regions and guide cell movement or swapping decisions to alleviate these hotspots. Techniques like cell bloating (increasing cell dimensions in congested areas during global placement) and whitespace injection (adding empty space in congested regions) are used to reduce cell density and indirectly alleviate routing pressure.
- **Post-Placement Congestion Minimization:** As mentioned before, net-centric post-processing techniques that focus on rerouting nets in congested areas by making small placement changes are explicitly designed to improve the fairness of routing demand distribution after the main placement phase.

The experimental results presented in some sources indicate that a two-step approach, involving initial wirelength minimization followed by a post-processing stage for congestion minimization (often using net-centric techniques), can be very effective in producing placements with significantly reduced congestion. This suggests that while global wirelength optimization provides a good starting point for overall routing demand reduction, a subsequent focus on local congestion hotspots and achieving a fairer distribution of routing resources is crucial for achieving high-quality, routable designs.

VI. A FAIRNESS-CENTRIC PLACER: THE NNS APPROACH

The preceding sections argued that global placement should optimize the *worst* connection rather than the total, and that doing so is a fairness property. This section shows that the principle is realizable in a compact, exact algorithm, following the open-source *No-Nonsense* (NNS) placer [46]. NNS deliberately avoids the smooth convex approximations that dominate analytical placement: it keeps a linear, non-smooth wire-length cost and solves the resulting min-max problem exactly with a parametric minimum-cost-flow engine. The resulting optimizer needs no floating-point arithmetic when a linear cost model is assumed, and it accommodates monotone (in particular concave) cost models that better match fixed FPGA fabrics.

A. Objective: Minimize the Worst Wire Length

Let the netlist connect modules by two-terminal arcs E . A placement assigns coordinates (x_v, y_v) to every module

v . Whereas a traditional placer minimizes a sum, NNS minimizes the maximum:

$$\min_{\text{placement}} \max_{(u,v) \in E} (c_x |x_u - x_v| + c_y |y_u - y_v|),$$

where c_x and c_y scale the per-axis wire costs. This is a min-max (bottleneck) objective: it bounds *every* connection, so no single net can be sacrificed for the average – exactly the fairness requirement argued above. Because it is a maximum of linear terms the objective remains convex, but it is non-smooth. More generally, the linear terms may be replaced by any monotone per-axis cost function $m(\cdot)$, which is how the placer targets the concave wire costs of real FPGA routing.

B. Per-Axis Optimization and Difference Constraints

NNS optimizes one axis at a time while the other axis is frozen, an alternating-direction scheme [47], [48]. Let q_v denote the coordinate of v on the axis under optimization, and treat the perpendicular coordinates as constants. For a trial *radius* r , the wire-length budget allowed on this axis, the placement is feasible on this axis if and only if every arc meets its budget:

$$q_v - q_u \leq \text{cost}((u, v), r), \quad \forall (u, v) \in E.$$

These are difference constraints; the system is feasible exactly when its constraint graph contains no negative cycle. Increasing r relaxes the constraints, so the smallest feasible radius is governed by the tightest cycle. Writing c_{uv} for the (integer) cost of arc (u, v) , the minimum feasible radius is the *minimum cycle ratio*

$$r^* = \max_{\text{cycles } \kappa} \frac{\sum_{(u,v) \in \kappa} c_{uv}}{|\kappa|},$$

which is the maximum cycle mean of the graph. The worst-wire-length bottleneck therefore reduces exactly to a minimum-cycle-ratio / parametric minimum-cost-flow problem, for which efficient optimum-cycle-mean algorithms are well studied [49], [50].

C. Howard's Algorithm and the Minimum Cycle Ratio

NNS solves the parametric problem by Howard's policy-iteration method [51], [52], as provided by the `digraphx` library [53]. In its parametric form the constraints become

$$q_v - q_u \leq \text{cost}(u, v) - r \tau_{uv},$$

where τ_{uv} is a per-arc “time” factor. The solver (i) relaxes the potentials q in a Bellman-Ford manner, (ii) inspects the predecessor/successor policy for cycles, (iii) raises r to the *zero-cancel* ratio of the critical cycle, and (iv) alternates the search direction until no further improvement is possible.

Each axis pass returns the exact worst-wire-length budget r^* on that axis.

Keeping the distance domain integral is what makes the method both exact and fast: with $r = n/d$, the per-arc distance collapses to a single integer floor-division,

$$\text{dist}(e, r) = \left\lfloor \frac{n - c d}{\delta d} \right\rfloor,$$

so an entire Howard sweep is executed with integer arithmetic while the ratio itself stays an exact rational.

D. Legalization by Minimum-Weight Bipartite Matching

Optimization alone lets several modules drift to the same coordinate. Legalization restores a one-module-per-site placement, line by line. Modules that share a coordinate on the perpendicular axis are collected into a bucket; a bipartite graph is built between the bucket's modules and the candidate sites within a growing window; and the edge weight is the *change in worst wire length* if a module is moved to that site. A minimum-weight full matching then assigns every module a distinct site:

$$\min_{\mu} \sum_v \Delta \text{worst}(v \rightarrow \mu(v)).$$

If no full matching exists within the local window, the window grows; a global fallback offers every free site on the line, so legalization fails only when the grid genuinely lacks capacity. The matching is solved directly with a linear-assignment routine (Hungarian method) [53], preserving the reference tie-breaking order so the placer's trajectory, and therefore its result, is deterministic. This is an instance of minimum-total-movement legalization, which can also be formulated as a minimum-cost flow [54]; here it is specialized to the min-max objective.

E. I/O Pad Ring Assignment

I/O pads must lie on the border ring around the core. For each pad, NNS chooses the side and coordinate nearest its connected modules, respecting a per-side capacity, and then legalizes the pads along each edge with the same matching machinery used for cells. Pads and cells share the per-line capacity, so a cap on the core must be applied consistently to the I/O ring as well.

F. The Outer Optimization Loop

One round of NNS runs Howard's algorithm on x , legalizes on y , and snaps the pads, then runs Howard on y , legalizes on x , and snaps the pads again. The worst wire length is recomputed; a round that does not improve it is rejected and the placement is rolled back to the previous snapshot, which also terminates the loop. This monotone acceptance guarantees that the output is never worse than its input and supplies a natural convergence criterion.

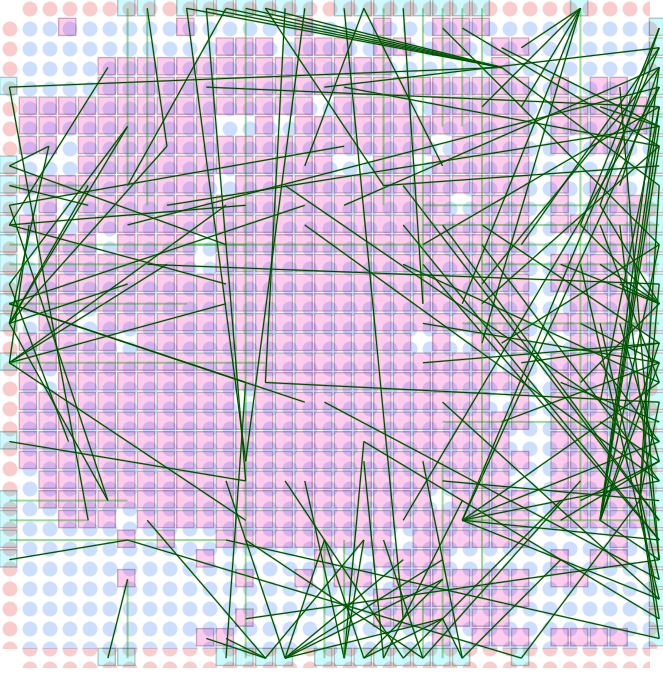


Figure 1: Final legal placement produced by NNS for the benchmark netlist **p1** on a 32x32 grid (752 cells, 81 I/O pads; worst wire length 1480). Blue squares are core cells, red squares are pads on the border ring, and green lines are the pad-to-module connections.

G. Implementation

NNS is implemented as a thin Python library on top of the sibling packages **digraphx** (parametric flow and negative-cycle detection), **netlistx** (netlist I/O), **physdes** (rectilinear geometry and routing), and **mywheel** [46], [53], [55], [56]. Its structure is organized around a few classic design patterns: a *Strategy* for the legalizer, a *Memento* for snapshot and rollback, an *Adapter* bridging the graph cost model to the solver, and a *Facade* for the top-level run. Treating bit-identical output on a fixed seed-grid matrix as a regression oracle allowed later refactorings to remove roughly 2,400 lines of duplicated infrastructure without changing a single placement.

VII. VISUALIZING FAIRNESS WITH CONGESTION MAPS

To judge whether a placement is *fair*, it is not enough to look at the wire-length objective; the routing demand it induces must be inspected. For every net, a global router builds an orthogonal routing tree, and each tree branch increments a counter for every horizontal or vertical grid-cut segment it crosses between adjacent lattice points. Per core cell, the x-direction congestion is the busiest horizontal cut immediately to its left or right, and the y-direction congestion is defined symmetrically; the combined map is their element-wise maximum. Each map is normalized so that its own busiest cut becomes 100 percent, and rendered with a green (0 percent) to yellow (50 percent) to red (100 percent) palette.

Splitting the map by direction exposes the two alternating sub-problems that NNS solves. Fig. 2 and Fig. 3 show the x- and y-direction maps for the 32x32 placement; their element-wise maximum is the combined map of Fig. 4, which is the quantity the min-max objective actually bounds.

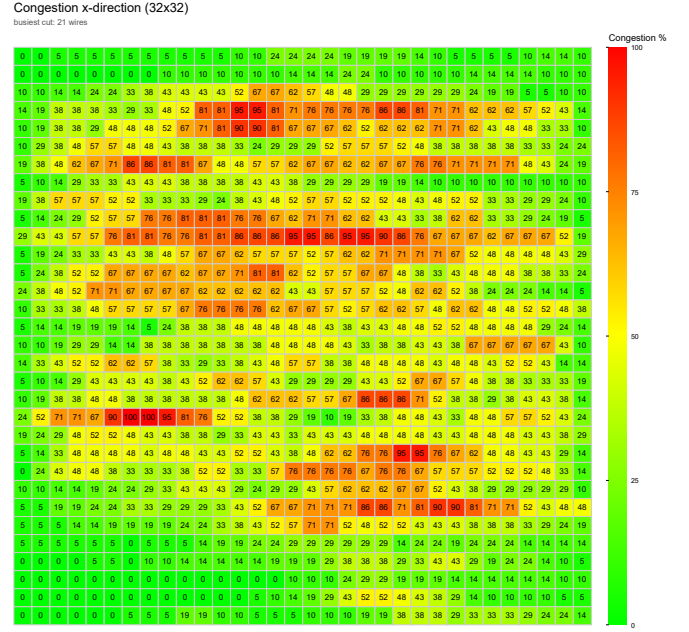


Figure 2: Congestion in the x-direction for the 32x32 placement, normalized to the busiest horizontal cut.

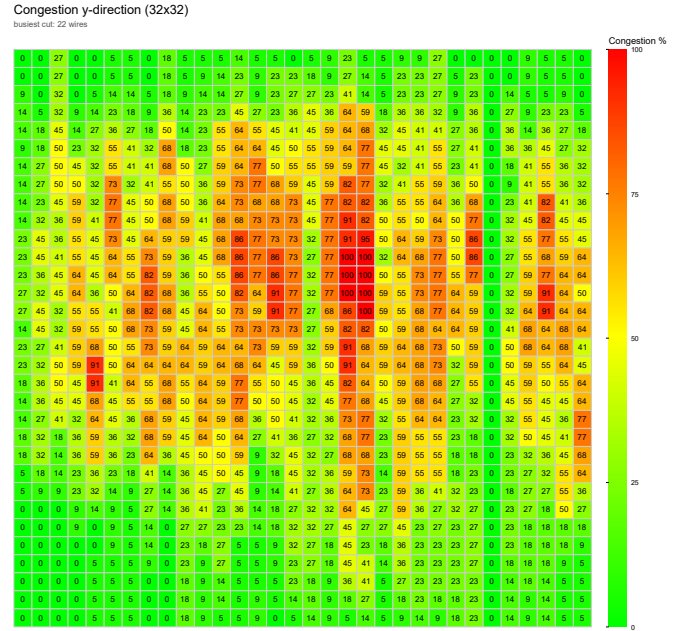


Figure 3: Congestion in the y-direction for the same placement, normalized to the busiest vertical cut.

These maps make the fairness argument concrete. A wirelength-optimal placement concentrates demand into a few bright, congested cuts, whereas a fairness-centric

Congestion combined (32x32)
busiest cut: 22 wires

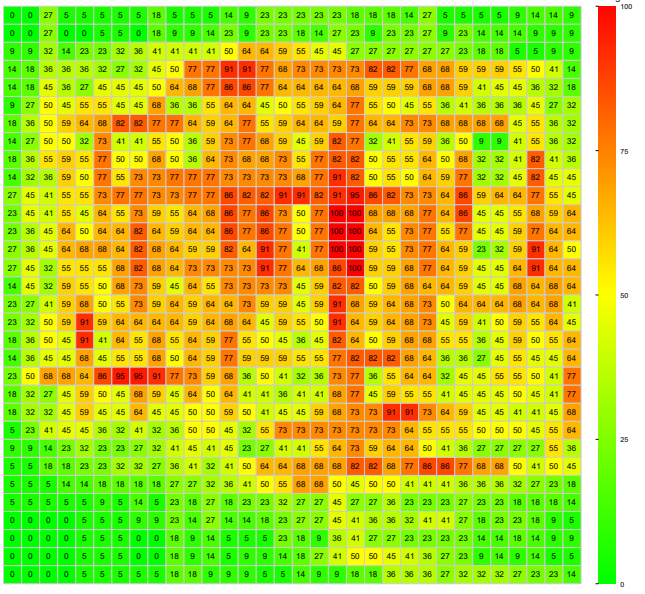


Figure 4: Combined congestion map (element-wise maximum of the x- and y-direction maps) for the 32x32 placement.

placement spreads demand more evenly, lowering the peak. Minimizing the worst wire length does not by itself guarantee a perfectly uniform map, but it removes the systematic blind spot of aggregate minimization: the worst connection can no longer be sacrificed for a better average.

VIII. EMPIRICAL EVALUATION

To show that the fairness objective is not only principled but affordable, the NNS implementation was exercised on the p1 benchmark: 752 core cells plus 81 I/O pads (833 modules), connected by 902 nets, of which 107 touch an I/O pad. Placements were computed on grids from 30x30 to 100x100 with a fixed random seed. Column 27 is reserved for DSP/SRAM, so a 30-wide grid offers only 29 usable cell columns. All reported placements are legal: no overlaps, no off-grid cells, and all pads on the border ring.

A. Convergence Across Grid Sizes

Table I summarizes the runs. Cell density is the number of cells divided by the usable core slots. The result illustrates the central tension of global placement: the *densest* grid (30x30, 86 percent) constrains the optimizer the most, so it cannot spread modules enough to shorten the longest nets and ends with the worst objective among the compact grids (1640). Roomier grids, by contrast, are not capacity-limited, and there the *search*, not the grid, limits quality.

B. Congestion Evenness Versus Density

Fig. 5 and Fig. 6 contrast the combined congestion maps of the tight 30x30 grid with the roomy 50x50 grid. On the dense grid the congestion is high and uneven because the

Table I: Convergence of NNS on the p1 benchmark (833 modules, 902 nets, fixed seed). “Rounds” is the number of accepted alternating x/y optimization rounds.

Grid	Usable slots	Cell density	Rounds	Worst
30x30	870	86%	2	1640
30x40	1160	65%	2	1440
32x32	992	76%	1	1480
40x30	1170	64%	2	1480
50x50	2450	31%	3	1600
100x100	9900	8%	2	2120

placer has little slack to redistribute cells; on the roomy grid the demand is lower and smoother. This is exactly the regime the fairness objective targets: rather than shrinking the total demand further, it caps the peak so that no region becomes a routing bottleneck.

Congestion combined (30x30)
busiest cut: 27 wires

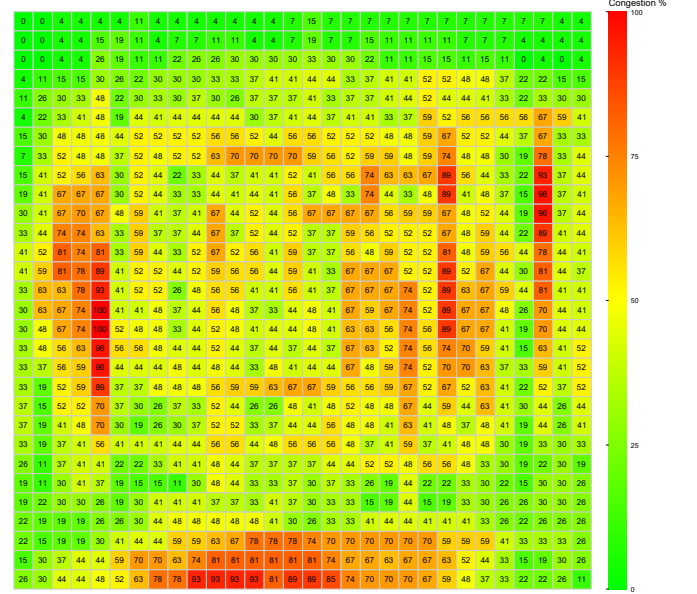


Figure 5: Combined congestion map on the tight 30x30 grid (86 percent cell density).

C. Computational Cost

The min-max objective and the exact rational arithmetic come at a cost, but a modest one. Profiling attributed most of the runtime to the Howard cost model and to legalization, and targeted, behavior-preserving optimizations reduced the total CPU time over a 30-scenario seed-grid matrix by a factor of roughly 4.6 without changing a single placement. Table II lists the headline figures; the same optimization was subsequently ported to C++ with further speed-ups. These figures are for p1 only; scaling to larger designs is discussed under Limitations below.

D. Preliminary Comparison with an Aggregate Baseline

To check whether the min-max objective trades aggregate quality for peak congestion, the repository ships a small comparison harness [46] that runs both placers on the same

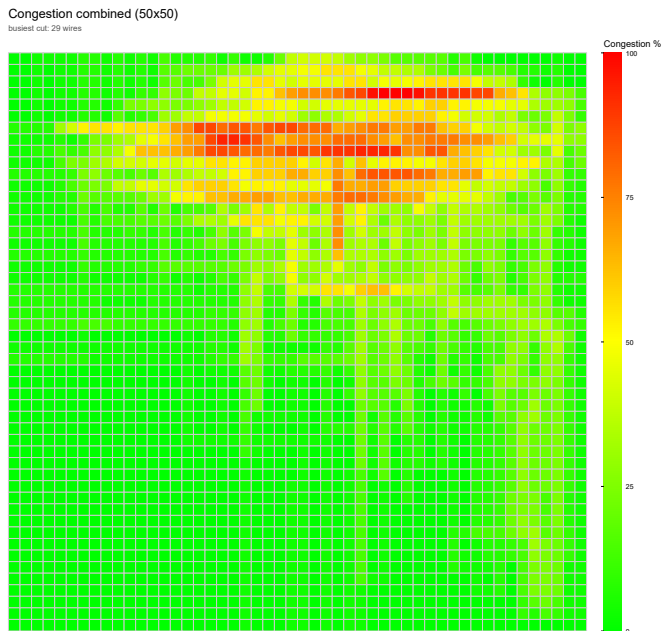


Figure 6: Combined congestion map on the roomy 50x50 grid (31 percent cell density).

Table II: Runtime of the fairness-centric placer before and after behavior-preserving optimization. Every change was validated against a bit-identical placement oracle, so the placement result is unchanged.

Optimization	Speed-up
Integer floor-division in the cost model	1.85x on hot callback
Direct linear-assignment solver (no graph)	2.6x per legalization
Gating infeasible assignment calls	7x per matching
Whole seed-grid matrix	4.57x total CPU

netlist and reports identical metrics: worst wirelength, total HPWL, the busiest routed grid cut, routed wirelength, legality, and runtime. The baseline minimizes squared wirelength on the same net-clique model, pins the pads evenly on the border ring, and legalizes row by row; it is a foil for the aggregate objective, not a state-of-the-art placer.

On `p1` (32x32) the min-max placer dominates on every metric. On `ibm01` (12,506 cells, 120x120, eight rounds) the picture is mixed and instructive: the min-max placer again produces the lower peak cut, but its worst wirelength and HPWL are higher and it is about four times slower. That is the coordinate-descent stall anticipated above – with a fixed iteration budget the alternating scheme has not yet spread the long nets that set the bottleneck. These results are preliminary (one seed, one simple baseline, proxy rather than sign-off routing), but they show both that the fairness objective lowers peak congestion and that closing the gap on large designs needs the multilevel and convergence work discussed next. Table III lists the numbers.

Table III: Preliminary comparison of the min-max placer against a quadratic aggregate baseline. “Worst” and “Peak” are in grid units; HPWL is in millions of grid units. One seed; routed wirelength is a proxy, not detailed-router sign-off.

Benchmark	Placer	Worst	HPWL	Peak
<code>p1</code> (32x32)	NNS	1600	0.57	26
<code>p1</code> (32x32)	quadratic	2120	0.58	33
<code>ibm01</code> (120x120)	NNS	8320	50.4	117
<code>ibm01</code> (120x120)	quadratic	7920	42.1	143

IX. FROM PLACEMENT TO GLOBAL ROUTING

Placement optimizes a proxy; routing decides the actual wires. To close the loop, each net of the final placement is passed to a rectilinear global router [56] that builds one tree per net, rooted at the net’s driver pin, and connects each sink within an allowed wire-length budget, inserting Steiner points to share trunks and save wire. Fig. 7 shows the routed version of the 50x50 placement: the straight proxy lines of Fig. 1 are replaced by orthogonal routing trees, and the cut crossings of those trees are precisely what the congestion maps of the previous section count.

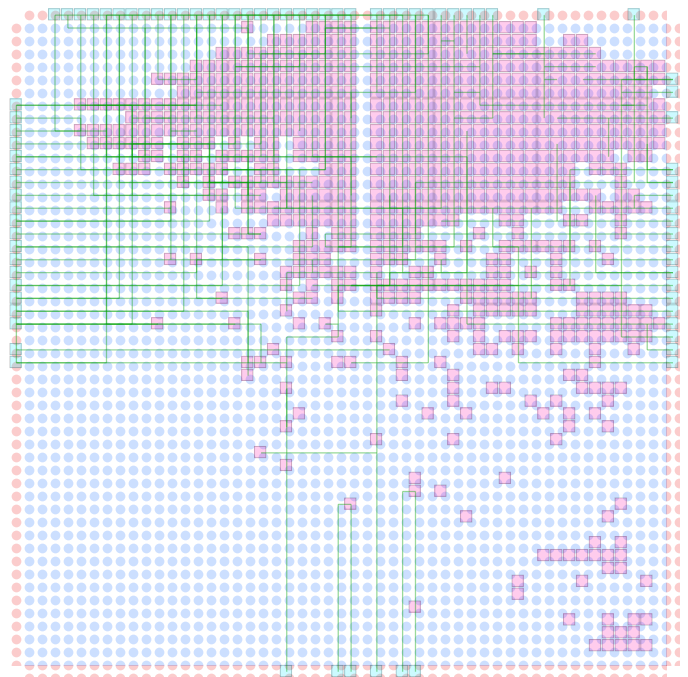


Figure 7: Routed placement on the 50x50 grid: orthogonal routing trees replace the straight proxy connections.

This two-stage view reinforces the paper’s thesis. The placer bounds the worst per-net wire length, the router materializes it, and the resulting congestion map reveals whether the demand is fairly distributed. When it is not, the placement objective – not just the total wire length – is the knob to turn.

X. RELATIONSHIP WITH OTHER PLACEMENT OBJECTIVES

Adopting a fairness-centric approach to global placement has implications for other important placement objectives, such as wirelength minimization, timing optimization, and power reduction. Understanding these relationships and potential trade-offs is crucial for developing effective placement methodologies.

A. Interaction with Wirelength Minimization

As discussed earlier, minimizing total wirelength and minimizing local congestion are not always mutually exclusive. While a purely wirelength-driven approach can lead to congestion, a placement with a more balanced routing demand distribution might have a slightly higher total wirelength. Therefore, a fairness-centric approach might involve accepting a small increase in total wirelength in exchange for a significant reduction in peak congestion levels, ultimately leading to better routability and potentially better final routed wirelength due to fewer detours. The goal is to find a balance where the routing demand is sufficiently low overall (thanks to wirelength minimization) and also distributed reasonably evenly across the chip (thanks to the fairness objective).

B. Implications for Timing-Driven Placement

Timing-driven placement aims to optimize circuit performance by minimizing critical path delays. This is often achieved by assigning weights to critical nets and encouraging the placer to shorten these nets. Incorporating fairness into timing-driven placement requires careful consideration. An aggressive focus on reducing congestion might inadvertently increase the length of critical paths, leading to timing violations. Conversely, solely focusing on timing might create or exacerbate congestion issues. Therefore, a successful approach might involve a multi-objective optimization that considers both timing criticality and congestion (or fairness in routing demand). This could involve assigning higher congestion penalties in regions containing critical nets or using timing-aware congestion estimation.

A fairness objective need not treat every net equally, which resolves much of the apparent conflict with timing-driven design. Max-min fairness is most naturally stated for *unweighted* agents, but weighted generalizations are standard in networking: generalized processor sharing allocates bandwidth in proportion to per-flow weights [57], and the same idea applies here. Give each net e a weight ω_e reflecting its timing criticality (or its switching activity, for power) and minimize the *weighted* bottleneck

$$\min_{\text{placement}} \max_{e \in E} \frac{\text{worst}(e)}{\omega_e},$$

so that a critical net may consume a larger wire budget before it binds the objective. Because the per-axis

subproblem is again a parametric minimum-cycle-ratio problem, the weights enter only through the per-arc costs and Howard's algorithm needs no structural change. A lexicographic variant, which optimizes the critical nets first and equalizes the remainder afterwards, is an alternative that trades one objective for a small hierarchy of them. Weighted max-min fairness therefore *reconciles* the fairness principle with timing- and power-driven placement rather than competing with it.

C. Impact on Power-Driven Placement

Power-driven placement aims to minimize power consumption, often by reducing wirelength (to decrease dynamic power) or by strategically placing cells with different power characteristics (e.g., in multi-voltage designs). A fairer distribution of cells achieved through a fairness-centric placement could potentially benefit power consumption by avoiding highly dense regions, which might lead to increased temperature and leakage power. Furthermore, shorter routed wirelengths (due to better routability) can also contribute to lower dynamic power consumption. In multi-voltage designs, the interaction between fairness in cell distribution and the placement of cells near their respective voltage sources would need to be carefully managed.

Overall, integrating fairness into global placement requires a holistic approach that considers its interplay with other crucial design objectives. The optimal balance between fairness and other metrics might depend on the specific characteristics and requirements of the target design.

XI. CHALLENGES AND FUTURE DIRECTIONS

While fairness-centric global placement offers a promising direction for addressing the critical issue of routing congestion in VLSI design, several challenges remain, and further research is needed to realize its full potential.

A. Computational Complexity

Directly optimizing for fairness metrics, such as minimizing the maximum congestion, can be computationally more complex than minimizing the sum of costs like total wirelength. Algorithms need to efficiently estimate congestion, identify the most congested regions, and make placement decisions that effectively reduce peak congestion without significantly degrading other objectives. Developing scalable and efficient algorithms for fairness-centric global placement, especially for very large-scale designs, is a significant challenge.

B. Scaling to Large Designs

Two ingredients limit NNS as designs grow. First, the outer loop is a block coordinate descent: each pass solves one axis exactly while the other is frozen, and a round is accepted only if the worst wirelength improves. On roomy grids this monotone acceptance stalls early – the 100x100 run of Table I is limited by the *search*, not by capacity, because

a single long net dominates the bottleneck and no axis-aligned move shortens it. Multilevel (clustering) placement, which coarsens the netlist before optimizing and refines afterwards [58], [59], and randomized restarts or a joint two-axis step are natural remedies. Second, exact parametric flow and matching are heavier per node than the gradient kernels of analytical placers; an efficient implementation must exploit sparsity, warm-start the flow from the previous round, and parallelize negative-cycle detection. The C++ port is a first step, but a rigorous scaling study – ideally with a GPU implementation in the spirit of DREAMPlace [29] – is needed before the approach can be positioned against production-grade engines.

C. Accurate and Efficient Congestion Estimation

The effectiveness of any congestion-driven (or fairness-centric) placement approach heavily relies on the accuracy and efficiency of the congestion estimation model used during the placement process. As noted, simpler models like bounding box routing provide a quick estimate but might lack accuracy, while more sophisticated global routing-based estimation can be more accurate but also more computationally expensive. Finding a good balance between accuracy and efficiency in congestion estimation during global placement remains an ongoing challenge.

D. Integration with Mixed-Size Placement

Modern VLSI designs often include a mix of standard cells and larger blocks (macros, IP cores). Integrating fairness-centric principles into mixed-size placement adds another layer of complexity. The placement of large, fixed-size macros significantly influences the available area for standard cell placement and the overall routing landscape. Developing fairness-aware placement algorithms that effectively handle both the placement of macros and the surrounding standard cells is an important area for future research [60].

E. Handling Multi-Layer Routing Resources

Modern ICs utilize multiple routing layers with varying characteristics. Congestion can occur on specific layers or across multiple layers. Fairness-centric placement needs to consider the availability and capacity of these multi-layer routing resources during congestion estimation and optimization. Developing sophisticated congestion models that accurately reflect the multi-layer routing environment is crucial.

F. Robust Benchmarks and Evaluation Metrics

To effectively evaluate and compare different fairness-centric global placement algorithms, the availability of robust benchmark circuits with known routing challenges and well-defined evaluation metrics is essential. Metrics should go beyond just total overflow and potentially consider the distribution of congestion, the impact on routed wirelength, and the overall routability of the final placement.

G. Future Research Directions

Future research in fairness-centric global placement can explore several promising directions:

- Development of novel and efficient algorithms that can directly optimize for fairness metrics like minimizing maximum congestion.
- Investigation of advanced machine learning techniques to predict congestion during early stages of placement and guide fairness-aware optimization.
- Exploration of tighter integration between global placement and early routing estimation to obtain more accurate congestion feedback for fairness optimization.
- Development of unified placement frameworks that can simultaneously address fairness, wirelength, timing, and power objectives in a balanced manner.
- Creation of new benchmark suites specifically designed to challenge and evaluate the effectiveness of congestion-driven and fairness-centric placement algorithms.
- Research into extending fairness principles to other stages of physical design, such as detailed placement and routing.

XII. LIMITATIONS AND THREATS TO VALIDITY

The study is a proof of concept, and several limitations bound how far its conclusions generalize.

- **Benchmark scale.** The controlled study uses a single synthetic benchmark, **p1** (833 modules), on grids up to 100x100 – two to three orders of magnitude smaller than the multi-million-cell designs that industrial placers target. A preliminary comparison harness also runs the larger **ibm01** benchmark (12,506 cells), but a single instance at each size cannot characterize a method’s average behavior; the results are evidence that the min-max objective is *realizable and affordable*, not that it is competitive at production scale.
- **No head-to-head comparison.** The paper reports a preliminary comparison only against a simple quadratic aggregate baseline (the preliminary subsection of the evaluation), on one seed and with proxy routed wirelength; the flow is not closed with a sign-off router, and NNS is not benchmarked against leading analytical or GPU placers (ePlace, RePlace, DREAMPlace, NTUPlace3) on a shared suite. Adopting the public ISPD/DAC placement and routability contests [43], [61] and reporting routed wirelength after a detailed router such as TritonRoute [62] would make the comparison reproducible and would test whether a lower peak congestion survives detailed routing.
- **Narrow metrics.** Only worst wirelength and cut-crossing congestion maps are reported; total and routed wirelength, timing, and power are not. Because minimizing the peak can raise the total, a complete evaluation must report both, together with peak and mean routing overflow.

- **Physical abstraction.** The model places cells on a uniform two-dimensional grid and legalizes row by row. It does not represent multi-layer metal stacks, via costs, or non-uniform track directions, and it assumes a single cell size with no fixed macros, so macro-dominated and mixed-size placement [60] is not exercised.
- **Scalability.** Howard’s parametric flow and bipartite-matching legalization are exact but heavier than the fast transform and sparse-linear-system kernels of analytical placers, and the reported 4.57x speedup was measured on p1 only. The growth of the flow graph with design size is not characterized.

These limitations do not undercut the paper’s central claim – that the worst connection, not the total, is the right quantity to bound – but they mark the distance between a realizable prototype and a production-ready placer.

XIII. CONCLUSION

The increasing complexity and density of modern VLSI circuits have elevated routing congestion to a critical concern in physical design. Traditional global placement objectives, primarily focused on minimizing total wirelength, often fail to adequately address the issue of localized congestion, leading to routability problems and performance degradation. This paper has argued for a shift towards **fairness-centric global placement**, where the goal is to achieve a more balanced distribution of routing demand across the chip by prioritizing the minimization of peak congestion levels.

The principle of **max-min fairness**, borrowed from network resource allocation, provides a compelling motivation for this approach, emphasizing the importance of ensuring a baseline level of routability across all regions of the design. While challenges remain in developing computationally efficient and accurate fairness-centric algorithms and integrating them with other placement objectives, the potential benefits in terms of improved routability, reduced design cycle times, and higher quality final layouts make this a crucial area for future research and development in VLSI physical design. By recognizing the inherent limitations of solely focusing on global minimization and embracing the concept of fairness in resource allocation, the VLSI design community can pave the way for more robust and scalable placement methodologies capable of handling the ever-increasing complexity of integrated circuits.

Beyond the principles, this paper reported a concrete realization: the open-source NNS placer, which reduces the worst-wire-length objective to a parametric minimum-cost-flow problem solved by Howard’s algorithm and legalizes with minimum-weight bipartite matching. Its placements yield congestion maps in which the fairness/routability trade-off is directly visible, and behavior-preserving optimizations keep the exact min-max objective affordable. We hope that the open implementation lowers the barrier to experimenting with fairness-centric objectives on real designs.

REFERENCES

- [1] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA, USA: Kluwer Academic Publishers, 1999.
- [2] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht, The Netherlands: Springer, 2011. doi: 10.1007/978-90-481-9591-6.
- [3] C. M. Fiduccia and R. M. Mattheyses, “A linear-time heuristic for improving network partitions,” in *Proc. Design automation conf. (DAC)*, 1982, pp. 175–181. doi: 10.1145/62882.62910.
- [4] K. M. Hall, “An r -dimensional quadratic placement algorithm,” *Management Science*, vol. 17, no. 3, pp. 219–229, 1970, doi: 10.1287/mnsc.17.3.219.
- [5] J. M. Kleinhans, G. Sigl, F. M. Johannes, and K. J. Antreich, “GORDIAN: VLSI placement by quadratic programming and slicing optimization,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 10, no. 3, pp. 356–365, 1991, doi: 10.1109/43.67789.
- [6] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang, “NTUplace3: An analytical placer for large-scale mixed-size designs with pre-placed blocks and density constraints,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 7, pp. 1228–1240, 2008, doi: 10.1109/tcad.2008.923063.
- [7] V. Betz and J. Rose, “VPR: A new packing, placement and routing tool for FPGA research,” in *Field-programmable logic and applications (FPL)*, 1997, pp. 213–222. doi: 10.1007/3-540-63465-7_226.
- [8] M. Gort and J. H. Anderson, “Analytical placement for heterogeneous FPGAs,” in *Proc. Int. Conf. Field programmable logic and applications (FPL)*, 2012, pp. 143–150. doi: 10.1109/fpl.2012.6339278.
- [9] N. Viswanathan and C. C.-N. Chu, “FastPlace: Efficient analytical placement using cell shifting, iterative local refinement, and a hybrid net model,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 24, no. 5, pp. 722–733, 2005, doi: 10.1109/tcad.2005.846365.
- [10] F. P. Brooks, “No silver bullet—essence and accidents of software engineering,” *Computer*, vol. 20, no. 4, pp. 10–19, 1987, doi: 10.1109/mc.1987.1663532.
- [11] M.-C. Kim, D.-J. Lee, and I. L. Markov, “SimPL: An effective placement algorithm,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 1, pp. 50–60, 2012, doi: 10.1109/tcad.2011.2170567.
- [12] W. C. Naylor, R. Donnelly, and L. Sha, “Non-linear optimization system and method for wire length and delay optimization for an automatic electric circuit placer.” U.S. Patent 6,301,693 B1, 2001. Available: <https://patents.google.com/patent/US6301693B1/en>

- [13] A. B. Kahng and Q. Wang, "Implementation and extensibility of an analytic placer," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 24, no. 5, pp. 734–747, 2005, doi: 10.1109/tcad.2005.846366.
- [14] J. Lu *et al.*, "ePlace: Electrostatics-based placement using fast Fourier transform and Nesterov's method," *ACM Trans. Design Automation of Electronic Systems*, vol. 20, no. 2, pp. 1–34, 2015, doi: 10.1145/2699873.
- [15] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang, "RePLace: Advancing solution quality and routability validation in global placement," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 9, pp. 1717–1730, 2019, doi: 10.1109/tcad.2018.2859220.
- [16] T. F. Chan, J. Cong, J. R. Shinnerl, K. Sze, and M. Xie, "mPL6: Enhanced multilevel mixed-size placement," in *Proc. Int. Symp. Physical design (ISPD)*, 2006, pp. 212–214. doi: 10.1145/1123008.1123055.
- [17] M. A. Breuer, "A class of min-cut placement algorithms," in *Proc. 14th design automation conf. (DAC)*, 1977, pp. 284–290.
- [18] B. W. Kernighan and S. Lin, "An efficient heuristic procedure for partitioning graphs," *Bell System Technical Journal*, vol. 49, no. 2, pp. 291–307, 1970, doi: 10.1002/j.1538-7305.1970.tb01770.x.
- [19] A. E. Dunlop and B. W. Kernighan, "A procedure for placement of standard-cell VLSI circuits," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 4, no. 1, pp. 92–98, 1985, doi: 10.1109/tcad.1985.1270101.
- [20] C. J. Alpert and A. B. Kahng, "Recent directions in netlist partitioning: A survey," *Integration, the VLSI Journal*, vol. 19, no. 1–2, pp. 1–81, 1995, doi: 10.1016/0167-9260(95)00008-4.
- [21] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983, doi: 10.1126/science.220.4598.671.
- [22] C. Sechen and A. Sangiovanni-Vincentelli, "The TimberWolf placement and routing package," *IEEE J. Solid-State Circuits*, vol. 20, no. 2, pp. 510–522, 1985, doi: 10.1109/jssc.1985.1052337.
- [23] S. Goto, "An efficient algorithm for the two-dimensional placement problem in electrical circuit layout," *IEEE Trans. Circuits and Systems*, vol. 28, no. 1, pp. 12–18, 1981, doi: 10.1109/tcs.1981.1084903.
- [24] N. R. Quinn and M. A. Breuer, "A forced directed component placement procedure for printed circuit boards," *IEEE Trans. Circuits and Systems*, vol. 26, no. 6, pp. 377–388, 1979, doi: 10.1109/tcs.1979.1084652.
- [25] H. Eisenmann and F. M. Johannes, "Generic global placement and floorplanning," in *Proc. 35th design automation conf. (DAC)*, 1998, pp. 269–274. doi: 10.1145/277044.277119.
- [26] W. E. Donath *et al.*, "Timing driven placement using complete path delays," in *Proc. 27th ACM/IEEE design automation conf. (DAC)*, 1990, pp. 84–89. doi: 10.1145/123186.123232.
- [27] A. E. Caldwell, A. B. Kahng, and I. L. Markov, "Can recursive bisection alone produce routable placements?" in *Proc. 37th design automation conf. (DAC)*, 2000, pp. 477–482. doi: 10.1145/337292.337549.
- [28] C. J. Alpert, T. F. Chan, A. B. Kahng, I. L. Markov, and P. Mulet, "Faster minimization of linear wirelength for global placement," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 1, pp. 3–13, 1998, doi: 10.1109/43.673628.
- [29] Y. Lin *et al.*, "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 4, pp. 748–761, 2021, doi: 10.1109/tcad.2020.3003843.
- [30] Y. Lin, W. Li, J. Gu, H. Ren, B. Khailany, and D. Z. Pan, "ABCDPlace: Accelerated batch-based concurrent detailed placement on multithreaded CPUs and GPUs," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 5083–5096, 2020, doi: 10.1109/tcad.2020.2971531.
- [31] A. Mirhoseini *et al.*, "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, 2021, doi: 10.1038/s41586-021-03544-w.
- [32] A. B. Kahng, R. Varadarajan, and Z. Wang, "HierRTLMP: A hierarchical automatic macro placer for large-scale complex IP blocks," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1552–1565, 2024, doi: 10.1109/tcad.2023.3346284.
- [33] G. Huang *et al.*, "Machine learning for electronic design automation: A survey," *ACM Trans. Design Automation of Electronic Systems*, vol. 26, no. 5, pp. 1–46, 2021, doi: 10.1145/3451179.
- [34] A. B. Kahng, "Machine learning for CAD/EDA: The road ahead," *IEEE Design & Test*, vol. 40, no. 1, pp. 8–16, 2023, doi: 10.1109/mdat.2022.3161593.
- [35] A. Marquardt, V. Betz, and J. Rose, "Timing-driven placement for FPGAs," in *Proc. ACM/SIGDA int. Symp. Field programmable gate arrays (FPGA)*, 2000, pp. 203–213. doi: 10.1145/329166.329208.
- [36] Z. Abuowaimer *et al.*, "GPlace3.0: Routability-driven analytic placer for UltraScale FPGA architectures," *ACM Trans. Design Automation of Electronic Systems*, vol. 23, no. 5, pp. 1–33, 2018, doi: 10.1145/3233244.
- [37] W. Li, S. Dhar, and D. Z. Pan, "UTPlaceF: A routability-driven FPGA placer with physical and congestion aware packing," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 4, pp. 869–882, 2018, doi: 10.1109/tcad.2017.2729349.

- [38] K. E. Murray *et al.*, “VTR 8: High-performance CAD and customizable FPGA architecture modelling,” *ACM Trans. Reconfigurable Technology and Systems*, vol. 13, no. 2, pp. 1–55, 2020, doi: 10.1145/3388617.
- [39] M. A. Elgammal, K. E. Murray, and V. Betz, “RLPlace: Using reinforcement learning and smart perturbations to optimize FPGA placement,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 8, pp. 2532–2545, 2022, doi: 10.1109/tcad.2021.3109863.
- [40] U. Brenner and A. Rohe, “An effective congestion driven placement framework,” in *Proc. Int. Symp. Physical design (ISPD)*, 2002, pp. 6–11. doi: 10.1145/505388.505391.
- [41] C. Li, M. Xie, C.-K. Koh, J. Cong, and P. H. Madden, “Routability-driven placement and white space allocation,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 5, pp. 858–871, 2007, doi: 10.1109/tcad.2006.884575.
- [42] X. He, T. Huang, L. Xiao, H. Tian, and E. F. Y. Young, “Ripple: A robust and effective routability-driven placer,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 10, pp. 1546–1556, 2013, doi: 10.1109/tcad.2013.2265371.
- [43] N. Viswanathan, C. J. Alpert, C. Sze, Z. Li, G.-J. Nam, and J. A. Roy, “The ISPD-2011 routability-driven placement contest and benchmark suite,” in *Proc. Int. Symp. Physical design (ISPD)*, 2011, pp. 141–146. doi: 10.1145/1960397.1960429.
- [44] D. P. Bertsekas and R. G. Gallager, *Data networks*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1992.
- [45] E. L. Hahne, “Round-robin scheduling for max-min fairness in data networks,” *IEEE J. Selected Areas in Communications*, vol. 9, no. 7, pp. 1024–1039, 1991, doi: 10.1109/49.103550.
- [46] W.-S. Luk, “nnsplace: An affordable fairness-centric placement library.” 2024. Available: <https://github.com/luk036/nnsplace>
- [47] A. B. Kahng, S. Mantik, and I. L. Markov, “Min-max placement for large-scale timing optimization,” in *Proc. Int. Symp. Physical design (ISPD)*, 2002, pp. 143–148. doi: 10.1145/505388.505423.
- [48] J. Cong, A. B. Kahng, G. Robins, M. Sarrafzadeh, and C. K. Wong, “Provably good performance-driven global routing,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 11, no. 6, pp. 739–752, 1992, doi: 10.1109/43.137519.
- [49] R. M. Karp, “A characterization of the minimum cycle mean in a digraph,” *Discrete Mathematics*, vol. 23, no. 3, pp. 309–311, 1978, doi: 10.1016/0012-365X(78)90011-0.
- [50] A. Dasdan, S. S. Irani, and R. K. Gupta, “Efficient algorithms for optimum cycle mean and optimum cost to time ratio problems,” in *Proc. 36th ACM/IEEE design automation conf. (DAC)*, 1999, pp. 37–42. doi: 10.1145/309847.309862.
- [51] R. A. Howard, *Dynamic programming and markov processes*. Cambridge, MA, USA: MIT Press, 1960.
- [52] A. Dasdan, “Experimental analysis of the fastest optimum cycle ratio and mean algorithms,” *ACM Trans. Design Automation of Electronic Systems*, vol. 9, no. 4, pp. 385–418, 2004, doi: 10.1145/1027084.1027085.
- [53] W.-S. Luk, “digraphx: Graph, negative-cycle and parametric minimum-cost-flow algorithms.” 2024. Available: <https://github.com/luk036/digraphx>
- [54] U. Brenner and J. Vygen, “Legalizing a placement with minimum total movement,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 23, no. 12, pp. 1597–1613, 2004, doi: 10.1109/tcad.2004.836733.
- [55] W.-S. Luk, “netlistx: Netlist i/o and graph-algorithm utilities.” 2024. Available: <https://github.com/luk036/netlistx>
- [56] W.-S. Luk, “physdes: Rectilinear geometry and global-routing primitives.” 2024. Available: <https://github.com/luk036/physdes-py>
- [57] A. K. Parekh and R. G. Gallager, “A generalized processor sharing approach to flow control in integrated services networks: The single-node case,” *IEEE/ACM Trans. Networking*, vol. 1, no. 3, pp. 344–357, 1993, doi: 10.1109/90.234856.
- [58] C. J. Alpert, J.-H. Huang, and A. B. Kahng, “Multilevel circuit partitioning,” in *Proc. 34th design automation conf. (DAC)*, 1997, pp. 530–533. doi: 10.1145/266021.266275.
- [59] G.-J. Nam, S. Reda, C. J. Alpert, P. Villarrubia, and A. B. Kahng, “A fast hierarchical quadratic placement algorithm,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 25, no. 4, pp. 678–691, 2006, doi: 10.1109/tcad.2006.870079.
- [60] S. N. Adya and I. L. Markov, “Consistent placement of macro-blocks using floorplanning and standard-cell placement,” in *Proc. Int. Symp. Physical design (ISPD)*, 2002, pp. 12–17. doi: 10.1145/505388.505392.
- [61] G.-J. Nam, C. J. Alpert, P. Villarrubia, B. Winter, and M. Yildiz, “The ISPD2005 placement contest and benchmark suite,” in *Proc. Int. Symp. Physical design (ISPD)*, 2005, pp. 216–220. doi: 10.1145/1055137.1055182.
- [62] A. B. Kahng, L. Wang, and B. Xu, “TritonRoute: The open-source detailed router,” *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 3, pp. 547–559, 2021, doi: 10.1109/tcad.2020.3003234.